

CloudwebAI Helpdesk

Developer Documentation

v1.0 — June 2026

Full-stack SaaS IT Helpdesk with AI-assisted ticket resolution, ITIL-compliant workflow, role-based access and email notifications.

Layer	Technology	Version
Frontend	Next.js + TypeScript + Tailwind CSS	14+
Backend	Python FastAPI + SQLAlchemy	Latest
Database	PostgreSQL	18
Auth	JWT (python-jose + bcrypt)	-
Email	SMTP via smtplib	-
AI (planned)	OpenClaw Foundation + RAG + FAISS	-
Hosting	Hostinger VPS (Kali Linux)	EU

Table of Contents

1. Project Structure
2. Prerequisites & Setup
3. Environment Variables
4. Database Schema
5. Backend API Reference
6. Frontend Pages & Routes
7. Authentication Flow
8. Email Notifications
9. ITIL Workflow
10. Role-Based Access Control
11. Git Workflow
12. Deployment (VPS)
13. Roadmap

1. Project Structure

```
CloudwebAI-helpdesk/
├── frontend/                                # Next.js 14 App Router
│   ├── app/
│   │   ├── layout.tsx                      # Root layout
│   │   ├── page.tsx                       # Redirects to /login
│   │   ├── login/page.tsx                 # Login page
│   │   ├── customer/page.tsx             # Customer ticket view
│   │   ├── tickets/page.tsx              # Agent ticket list
│   │   ├── dashboard/page.tsx            # Agent dashboard
│   │   ├── admin/page.tsx                # Admin panel
│   │   └── components/
│   │       └── Topbar.tsx                 # Shared navigation bar
│   └── package.json
├── backend/                                # Python FastAPI
│   ├── main.py                            # App entry + all endpoints
│   ├── database.py                       # SQLAlchemy engine + session
│   ├── models.py                         # ORM models (all tables)
│   ├── .env                             # Secrets (NOT in git)
│   ├── requirements.txt                  # Python dependencies
│   └── docs/                             # IT documents for RAG
├── FRONTEND.md                           # Frontend documentation
├── DATABASE.md                           # Database documentation
├── PRODUCT.md                            # Product description
├── IDEAT.md                              # Feature ideas backlog
└── .gitignore
```

2. Prerequisites & Setup

Requirements

- Node.js 20+ and npm
- Python 3.11+
- PostgreSQL 18
- Git

Backend Setup

```
cd backend
python -m venv venv

# Windows
venv\Scripts\activate

# Mac/Linux
source venv/bin/activate

pip install -r requirements.txt
```

Frontend Setup

```
cd frontend
npm install
npm run dev
# → http://localhost:3000
```

Run Backend

```
cd backend
venv\Scripts\activate      # Windows
uvicorn main:app --reload
# → http://localhost:8000
# → http://localhost:8000/docs (Swagger UI)
```

Database Setup

```
# Connect to PostgreSQL
psql -U postgres

# Create database
CREATE DATABASE helpdesk;
\c helpdesk

# Run schema (tables are auto-created by SQLAlchemy
# on first backend start via Base.metadata.create_all)

# Create test users (password: admin123)
# Use pgAdmin or psql to run INSERT statements
# See DATABASE.md for full setup SQL
```

3. Environment Variables

Create backend/.env (never commit this file):

```
# Database
DATABASE_URL=postgresql://postgres:PASSWORD@localhost:5432/helpdesk

# JWT Authentication
SECRET_KEY=your-super-secret-key-here
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=480

# Email (Gmail SMTP)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your@gmail.com
SMTP_PASSWORD=your-16-char-app-password
SMTP_FROM=your@gmail.com
```

Gmail App Password Setup

1. Go to myaccount.google.com
2. Enable 2-Step Verification
3. Go to myaccount.google.com/apppasswords
4. Create new app password (name: 'helpdesk')
5. Copy the 16-character code (no spaces) to SMTP_PASSWORD

Frontend Environment (optional)

```
# frontend/.env.local
NEXT_PUBLIC_API_URL=http://localhost:8000

# Production:
NEXT_PUBLIC_API_URL=https://api.yourdomein.com
```

4. Database Schema

9 tables. All IDs are UUID. See DATABASE.md for full details.

Table	Purpose	Key Fields
companies	Client companies	id, name, contact_email, phone
users	All users (customers/agents/admins)	id, company_id, email, role, password_hash
tickets	Support tickets	id, customer_id, agent_id, status, priority, ticket_type
time_logs	Time tracking per ticket	id, ticket_id, agent_id, started_at, stopped_at, seconds
comments	Ticket messages & internal notes	id, ticket_id, user_id, content, is_internal
email_logs	Sent email history	id, ticket_id, to_email, type, sent_at
ai_suggestions	AI response tracking	id, ticket_id, original, final, outcome
solution_kb	AI learning knowledge base	id, problem_keywords, solution_text, times_used

rag_feedback	RAG document relevance	id, suggestion_id, doc_name, was_helpful
--------------	------------------------	--

Ticket Status Flow (ITIL)

Uusi → Luokiteltu → Kasittelyssa → Odottaa → Ratkaistu → Suljettu



Timer logic:

```
status = 'Kasittelyssa' → timer START
status = 'Odottaa' → timer PAUSE
status = 'Ratkaistu' → timer STOP + save total
status = 'Suljettu' → timer STOP
```

5. Backend API Reference

Base URL: <http://localhost:8000> | Swagger UI: </docs>

All protected endpoints require ?token=JWT_TOKEN as query parameter.

Method	Endpoint	Auth	Description
GET	/health	No	Health check
POST	/auth/login	No	Login → returns JWT token + role
GET	/tickets	Yes	List tickets (customers see only own)
POST	/tickets	Yes	Create new ticket
PUT	/tickets/{id}/status	Yes	Update status + timer logic
PUT	/tickets/{id}/agent	Yes	Assign agent to ticket
POST	/tickets/{id}/reply	Yes	Send reply (also triggers email)
GET	/tickets/{id}/comments	Yes	Get ticket comment history
POST	/tickets/{id}/comments	Yes	Add comment or internal note
GET	/users	Admin	List all users
POST	/users	Admin	Create user
PUT	/users/{id}	Admin	Update user details
POST	/users/{id}/reset-password	Admin	Reset user password
PUT	/users/{id}/active	Admin	Toggle user active/inactive
DELETE	/users/{id}	Admin	Delete user
GET	/companies	Yes	List companies
POST	/companies	Admin	Create company
DELETE	/companies/{id}	Admin	Delete company
GET	/agents	Yes	List agents (for dropdown)

Example: Login Request

POST /auth/login

Content-Type: application/json

```
{
  "email": "admin@cloudwebai.fi",
  "password": "admin123"
}
```

Response 200:

```
{
  "token": "eyJhbGciOi...",
  "role": "admin",
  "name": "Admin Kayttaja",
  "email": "admin@cloudwebai.fi",
  "id": "uuid-here"
}
```

Example: Create Ticket

POST /tickets?token=JWT_TOKEN
Content-Type: application/json

```
{  
  "title": "VPN ei toimi",  
  "description": "En paase VPN:lle...",  
  "priority": "Kiireellinen",  
  "ticket_type": "Incident"  
}
```

Response 200:

```
{  
  "id": "uuid-here",  
  "title": "VPN ei toimi",  
  "status": "Uusi"  
}
```

6. Frontend Pages & Routes

Route	File	Role	Description
/login	app/login/page.tsx	All	Login form, redirects by role
/customer	app/customer/page.tsx	customer	Own tickets + create ticket
/tickets	app/tickets/page.tsx	agent/admin	All tickets table with filters
/dashboard	app/dashboard/page.tsx	agent/admin	Ticket detail + reply + timer
/admin	app/admin/page.tsx	admin	User & company management

Shared Component: Topbar

```
// components/Topbar.tsx
// Usage:
<Topbar activePage="dashboard" />

// Props:
type Props = {
  activePage: 'tickets' | 'dashboard' | 'admin' | 'customer'
}

// Reads role from localStorage and shows navigation accordingly:
// admin   → Tiketit | Dashboard | Admin
// agent   → Tiketit | Dashboard
// customer → Omat tiketit
```

Auth Pattern (all pages)

```
// Standard auth pattern used in every page:
const [token, setToken] = useState<string | null>(null)

useEffect(() => {
  setToken(localStorage.getItem('token'))
}, [])

useEffect(() => {
  if (token === null) return // still loading
  if (!token) { router.push('/login'); return }
  fetchData()
}, [token])

// All API calls include token as query param:
fetch(`http://localhost:8000/tickets?token=${token}`)
```

7. Authentication Flow

1. User submits email + password to POST /auth/login
2. Backend verifies password with `bcrypt.checkpw()`
3. Backend creates JWT token:

```
payload = { "sub": user_id, "role": user.role, "exp": ... }  
token = jwt.encode(payload, SECRET_KEY, ALGORITHM)
```
4. Frontend stores in `localStorage`:

```
localStorage.setItem('token', data.token)  
localStorage.setItem('role', data.role)  
localStorage.setItem('name', data.name)
```
5. All subsequent requests include token as query param
6. Backend decodes and validates token on every request
7. Role-based access enforced per endpoint

User Roles

Role	Can Do
customer	Create tickets, view own tickets, see replies
agent	View all tickets, update status, reply, assign agents
admin	Everything agent can + user/company management

8. Email Notifications

Triggers

- Ticket created → confirmation email to customer
- Agent replies (`is_internal=False`) → notification email to customer

Implementation

```
# backend/main.py – send_email() function  
async def send_email(to: str, subject: str, body: str):  
    msg = MIMEMultipart('alternative')  
    msg['Subject'] = subject  
    msg['From'] = os.getenv('SMTP_FROM')  
    msg['To'] = to  
    msg.attach(MIMEText(body, 'html'))  
  
    server = smtplib.SMTP(SMTP_HOST, SMTP_PORT)  
    server.starttls()  
    server.login(SMTP_USER, SMTP_PASSWORD)  
    server.sendmail(SMTP_FROM, to, msg.as_string())  
    server.quit()  
  
# Called in:  
# - POST /tickets (after ticket created)  
# - POST /tickets/{id}/comments (if not internal)
```

9. ITIL Workflow

Ticket Types

- **Incident:** Something is broken — VPN not working, email down
- **Service Request:** New request — install software, create account
- **Problem:** Recurring issue — network drops every week
- **Change:** Infrastructure change — upgrade Windows, replace device

Status Transitions

Uusi (New)

```
■■→ Luokiteltu (Classified)    # Agent reviews + sets ticket_type
    ■■→ Kasittelyssa (In Progress) ■ TIMER STARTS
        ■■→ Odottaa (Waiting)    ■ TIMER PAUSED
        ■ ■■→ Kasittelyssa        ■ TIMER RESUMES
        ■■→ Ratkaistu (Resolved) ■ TIMER STOPS
            ■■→ Suljettu (Closed)
```

Timer Logic in Backend

```
# PUT /tickets/{id}/status
if new_status == "Kasittelyssa":
    # Create new TimeLog entry
    time_log = TimeLog(ticket_id=id, started_at=now())
    db.add(time_log)

elif new_status in ["Odottaa", "Ratkaistu", "Suljettu"]:
    # Find open TimeLog and close it
    log = db.query(TimeLog).filter(stopped_at == None).first()
    log.stopped_at = now()
    log.seconds = (log.stopped_at - log.started_at).total_seconds()

    if new_status in ["Ratkaistu", "Suljettu"]:
        # Sum all time logs → ticket.time_spent_seconds
        ticket.time_spent_seconds = sum(all_logs.seconds)
        ticket.resolved_at = now()
```

10. Role-Based Access Control

```
# Frontend – Topbar shows navigation by role:
role === 'admin'    → Tiketit | Dashboard | Admin
role === 'agent'    → Tiketit | Dashboard
role === 'customer' → Omat tiketit

# Frontend – Ticket visibility:
role === 'customer' → only own tickets (filtered by API)
role === 'agent'    → all tickets
role === 'admin'    → all tickets

# Backend – Ticket query:
if user.role == "customer":
    query = query.filter(Ticket.customer_id == user.id)
```

```
# Backend – Admin-only endpoints:  
if user.role != "admin":  
    raise HTTPException(403, "No permission")
```

11. Git Workflow

```
# Clone repository
git clone https://github.com/esatirkkonen/ai-helpdesk-.git
cd ai-helpdesk-
```

```
# Daily workflow
git add .
git commit -m "descriptive commit message"
git push
```

```
# Pull latest changes (second machine)
git pull
```

```
# .gitignore includes:
# backend/venv/
# backend/.env
# backend/faiss_index/
# frontend/node_modules/
# frontend/.next/
# frontend/.env.local
```

IMPORTANT: Never commit .env files!

On a new machine, create backend/.env manually with the correct values.

12. Deployment (VPS)

Target: Hostinger VPS running Kali Linux (EU datacenter)

VPS Setup

```
# Install dependencies
apt update
apt install -y nodejs npm python3 python3-pip postgresql git nginx
```

```
# Clone repo
cd /opt
git clone https://TOKEN@github.com/esatirkkonen/ai-helpdesk-.git
cd ai-helpdesk-
```

```
# Backend
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
# Create .env manually!
```

```
# Frontend
cd ../frontend
npm install
npm run build # Creates .next/ production build
```

Docker Compose (recommended)

```
# docker-compose.yml (to be created)
version: '3.8'
services:
  frontend:
    build: ./frontend
    ports: ["3000:3000"]

  backend:
    build: ./backend
    ports: ["8000:8000"]
    env_file: ./backend/.env

  postgres:
    image: postgres:18
    environment:
      POSTGRES_DB: helpdesk
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Nginx Reverse Proxy

```
# /etc/nginx/sites-available/cloudwebai
server {
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://localhost:3000;
    }

    location /api/ {
        proxy_pass http://localhost:8000/;
    }
}
```

13. Roadmap

Feature	Status	Priority
Ticket creation + CRUD	Done	P0
JWT Authentication	Done	P0
Role-based views	Done	P0
Email notifications	Done	P0
ITIL status flow	Done	P0
Comment history	Done	P0
Admin user management	Done	P0
ITIL ticket type classification	In Progress	P1
SLA tracking + countdown	In Progress	P1
Responsive design (mobile)	Planned	P1
Code refactoring to components	Planned	P2
Docker + VPS deployment	Planned	P1
AI integration (OpenClaw + RAG)	Planned	P2
Asset Agent (device info collection)	Planned	P2
Reports + PDF export	Planned	P2
Billing / time tracking export	Planned	P3
Email-to-ticket	Planned	P3
Reseller portal + white-label	Planned	P3
Stripe payment integration	Planned	P3